

Product Requirements Document

Hello World API Enhancement

1. Product Name

Hello World API

2. Purpose

The purpose of this API is to provide a simple endpoint that confirms the application is running correctly, while also serving as a training foundation for professional API development patterns.

This API will be used to practice:

- API routing
- Controllers
- DTOs
- Service layer design
- `ServiceResult<T>` pattern
- Validation
- Error handling
- Swagger testing

3. Current State

The API currently has a basic Hello World endpoint.

Existing Endpoint

```
GET /api/hello
```

Existing Response Example

```
"Hello World"
```

4. New Feature Request

Add an enhanced Hello World endpoint that accepts a user name and returns a personalized greeting response.

5. Business Requirement

As an API user, I want to send a name to the Hello World API so that I can receive a personalized greeting message.

6. User Story

As an API user,

I want to pass my name into the Hello World API,
so that I can receive a custom greeting message.

7. Functional Requirements

Requirement 1: Personalized Greeting Endpoint

The API shall expose a new endpoint that accepts a name as a route parameter.

```
GET /api/hello/{name}
```

Example Request

```
GET /api/hello/Franky
```

Example Successful Response

```
{
  "message": "Hello, Franky!"
}
```

8. Response DTO

Create a response DTO for the greeting response.

```
public class HelloResponseDto
{
    public string Message { get; set; } = string.Empty;
}
```

9. Validation Rules

The API shall validate the name value before returning a response.

Rule	Expected Result
Name is empty	Return 400 Bad Request
Name is whitespace	Return 400 Bad Request
Name is longer than 50 characters	Return 400 Bad Request
Name is valid	Return 200 OK

10. Error Response Examples

Empty or Whitespace Name

```
{
  "errors": [
    {
      "type": "Validation",
      "message": "Name is required."
    }
  ]
}
```

Name Too Long

```
{
  "errors": [
    {
      "type": "Validation",
      "message": "Name cannot be longer than 50 characters."
    }
  ]
}
```

```
}  
]  
}
```

11. Service Layer Requirement

Create a service interface and service implementation for the Hello World feature.

Interface Method

```
Task<ServiceResult<HelloResponseDto>> GetGreetingAsync(string name);
```

Service Responsibilities

- Accept a name value.
- Validate the name.
- Return validation errors when the name is invalid.
- Return a successful HelloResponseDto when the name is valid.
- Return ServiceResult<HelloResponseDto> instead of returning raw DTOs directly.

12. Controller Requirement

The controller should stay thin and should not contain business validation logic.

Example Controller Method

```
[HttpGet("{name}")]  
public async Task<IActionResult> GetGreeting(string name)  
{  
    var result = await _helloService.GetGreetingAsync(name);  
  
    return result.ToActionResult();  
}
```

13. ServiceResult Pattern Requirement

The feature should use the existing ServiceResult pattern.

Expected flow:

```
Controller calls service  
Service returns ServiceResult<T>  
Controller calls ToActionResult()  
ToActionResult converts the result into the correct HTTP response
```

Expected HTTP Mapping

Service Error Type	HTTP Response
Validation	400 Bad Request
NotFound	404 Not Found
Conflict	409 Conflict
Failure	500 Internal Server Error

14. Acceptance Criteria

Scenario 1: Valid Name

Given the user passes a valid name

When they call:

```
GET /api/hello/Franky
```

Then the API returns:

```
200 OK
```

And the response body is:

```
{  
  "message": "Hello, Franky!"  
}
```

Scenario 2: Blank Name

Given the user passes a blank or whitespace name

When the API processes the request

Then the API returns:

```
400 Bad Request
```

And the response contains:

```
Name is required.
```

Scenario 3: Name Too Long

Given the user passes a name longer than 50 characters

When the API processes the request

Then the API returns:

```
400 Bad Request
```

And the response contains:

```
Name cannot be longer than 50 characters.
```

15. Technical Tasks

- Create HelloResponseDto.
- Create IHelloService.
- Create HelloService.
- Add GetGreetingAsync(string name).
- Add validation for empty, whitespace, and long names.
- Return ServiceResult<HelloResponseDto> from the service.
- Register IHelloService and HelloService in Program.cs.
- Add GET /api/hello/{name} to HelloController.
- Use result.ToActionResult() in the controller.
- Test success and validation cases in Swagger.

16. Suggested Folder Structure

```
Controllers
    HelloController.cs

DTOs
    HelloResponseDto.cs

Services
    IHelloService.cs
    HelloService.cs

Common
    ServiceResult.cs
    ServiceError.cs
    ServiceErrorType.cs
    ServiceResultExtensions.cs
```

17. Out of Scope

- Database access
- Authentication
- Authorization
- Logging
- Application Insights
- POST endpoint
- Unit tests
- Deployment changes

18. Future Enhancements

After the first version is complete, the API can be expanded with a POST greeting endpoint.

```
POST /api/hello
```

Example Request Body

```
{
  "name": "Franky"
}
```

Example Response

```
{
  "message": "Hello, Franky!"
}
```

This future enhancement would introduce request DTOs and body validation.

19. Definition of Done

- The new endpoint exists.
- The endpoint returns a personalized greeting for valid names.
- Invalid names return validation errors.
- The service uses `ServiceResult<HelloResponseDto>`.
- The controller uses `ToActionResult()`.

- The controller does not contain business validation logic.
- The feature has been tested in Swagger.